

Deep Learning Laboratory (AI29204)

Endsem — 14/04/2026

Marks: 100

Time: 2-5

Instructions:

- **Answer Section A and any one question from Section B or C.**

Section-A [50]

Q1. Dataset:

EuroSAT (torchvision.datasets.EuroSAT): Satellite RGB images, 10 land-use classes (Highway, Forest, River, Industrial, etc.), 64×64 pixels.

Normal class: "Forest" , use all other 9 classes as anomalies (test only).

Task1: Data Loading and Spatial Pretext Setup (3 Marks)

- a. Load EuroSAT. Filter "Forest" samples as the normal class. Use all remaining classes as anomalies (reserved for the test set only).
- b. Normalize all images to $[-1, 1]$. Split the normal data into 80% train and 20% validation.
- c. Implement a custom Dataset class that, for each image, additionally returns a 2×2 jigsaw puzzle. Concretely: divide the 64×64 image into four 32×32 patches, apply a random permutation from the 24 possible orderings, and return both the shuffled patch tensor and the permutation index (0-23) as the label. This jigsaw label will be used as a self-supervised signal during training.

Note: Your Dataset class must return (original_image, shuffled_patches, permutation_index) as a tuple for each sample.

Task2: Visualisation (2 Marks)

- a. Plot 5 normal (Forest) samples and 5 anomalous samples side by side in a single figure. Label each image with its class name.
- b. For 2 of the normal samples, also show their jigsaw-shuffled version beneath the original. Label each patch position (1, 2, 3, 4) to indicate the applied permutation order.

Task 3: Architecture Design (10 Marks)

Design a VAE-GAN hybrid with the following three components:

Encoder (Shared Backbone)

- 3 Conv layers, channels [32, 64, 128], each followed by BatchNorm and LeakyReLU
- Final output: two linear projections to produce μ and $\log_var$ for a latent space of dimension 128
- SSL Head: An additional linear layer enc_jigsaw that takes the pre-latent flattened feature map and outputs logits over 24 permutation classes. This head is trained using the jigsaw pretext task.

Generator / Decoder

- Transpose Conv layers mirroring the encoder structure, reconstructing a 64×64 RGB image from the sampled latent vector z .
- Output activation: Tanh (to match the $[-1, 1]$ normalization range).

Discriminator

- 3-layer CNN outputting a single scalar (real vs. fake probability).
- Feature Extractor: The discriminator must expose its intermediate feature maps from layer 2. These will be used to compute the Feature Matching Loss.
- SSL Head: An additional linear layer `disc_jigsaw` on the layer-2 features, outputting logits over 24 permutation classes. The discriminator also participates in jigsaw pretext supervision.

Note: The `enc_jigsaw` and `disc_jigsaw` heads receive the same supervised jigsaw labels but learn representations from fundamentally different signals (reconstruction vs. discrimination). Think about what this implies for the quality of anomaly features each head captures.

Task 4: Training Loop (12 Marks)

Train for 25 epochs using the following multi-component loss functions. Use Adam optimizer for both Generator/Encoder and Discriminator with $\text{lr} = 2\text{e-}4$.

Generator / Encoder Loss:

$$L_G = L_{KL} + L_{adv} + \lambda_1 * L_{feat} + \lambda_2 * L_{jigsaw}$$

Where:

- L_{KL} : KL Divergence loss for the VAE bottleneck (enforces the latent space to be approximately Gaussian).
- L_{adv} : Binary Cross Entropy adversarial loss (Generator tries to fool the Discriminator).
- L_{feat} : L2 distance between the Discriminator's layer-2 feature maps of the real image vs. the reconstructed image (Feature Matching Loss).
- L_{jigsaw} : Cross-entropy loss on jigsaw permutation prediction, summed from both the `enc_jigsaw` head and the `disc_jigsaw` head.
- Hyperparameters: $\lambda_1 = 0.1$, $\lambda_2 = 0.5$

Discriminator Loss:

- Standard GAN BCE loss distinguishing real images from reconstructed (fake) images.

Track and plot the following per epoch:

- Generator Loss (total)
- Discriminator Loss
- KL Divergence value (watch for posterior collapse : $KL \rightarrow 0$ is a warning sign)
- Jigsaw classification accuracy on the validation set (from `enc_jigsaw`)

Task 5: Reconstruction Visualisation (3 Marks)

At epochs 1, 10, and 25, display a grid showing 4 normal samples alongside their corresponding reconstructions (original | reconstructed pairs).

In a Markdown cell below the grid, briefly comment (2–3 sentences): does the improvement in reconstruction quality across epochs correlate with the rise in jigsaw validation accuracy? What does this suggest about the role of the SSL pretext task in shaping the encoder's representations?

Task 6: Ablation Study (5 Marks)

Train the following two additional variants for 5 epochs each (a shorter run is sufficient for comparison):

- Variant A :No SSL: Standard VAE-GAN without the jigsaw loss (set $\lambda_2 = 0$). All other components remain identical.
- Variant B : SSL Only: Train only the Encoder with the jigsaw head (enc_jigsaw) using cross-entropy loss. No Generator, no Discriminator, no KL loss.

Report the following in a summary table:

Variant	Val. Reconstruction Loss	Jigsaw Val. Accuracy
Full VAE-GAN + SSL (Task 4)		
Variant A: No SSL		
Variant B: SSL only		

In a Markdown cell (3-4 sentences): State which variant performs best on validation reconstruction loss. Explain whether the spatial jigsaw pretext task helps or hurts the anomaly detection objective, and why. Connect your reasoning to what the encoder learns in each case.

Task 7: Anomaly Scoring with SSL Augmentation (7 Marks):

Implement and evaluate two anomaly scores on a test set of 200 normal + 200 anomalous samples.

Score 1 : Standard Anomaly Score:

$$\text{Score}(x) = \lambda * ||x - G(E(x))||_1 + (1 - \lambda) * D_{\text{feat}}(x, G(E(x)))$$

Where D_{feat} is the L2 distance between Discriminator layer-2 features of the real and reconstructed image. Use $\lambda = 0.5$.

Score 2 :SSL-Augmented Anomaly Score:

$$\text{Score_SSL}(x) = \text{Score}(x) + \gamma * (1 - p_{\text{correct}}(x))$$

Where $p_{\text{correct}}(x)$ is the softmax probability assigned by enc_jigsaw to the correct jigsaw permutation for image x , and $\gamma = 0.3$.

Produce a box plot showing the distribution of $\text{Score}(x)$ and $\text{Score_SSL}(x)$ for normal vs. anomalous samples side by side (4 boxes total). Report which score gives better class separation.

Task 8: Evaluation and Written Reasoning (8 Marks):

- ROC-AUC: Plot the ROC curve for both $\text{Score}(x)$ and $\text{Score_SSL}(x)$ on the same axes. Report the AUC for each.
- Threshold Analysis: At the decision threshold that achieves 90% Recall on anomalies, report Precision and F1-score for both scoring methods.

- c. Top Anomaly Visualization: Display the 5 anomalous samples with the highest $\text{Score_SSL}(x)$. For each, show: the original image, its reconstruction, and its score value. Label with the true class name.
- d. [Written : 2 Marks] In a Markdown cell: your model was trained exclusively on Forest images. Two of the anomalous classes are "River" and "Highway." Based on your actual box plot and top anomaly visualizations from parts (a)-(c), does your model assign higher Score_SSL to River or to Highway images? Provide a hypothesis, grounded in what spatial textures and structures the encoder has learned from Forest images, for why this ordering makes sense.

Section-B [50]

Q2. Task A: Model Architecture [5 Mark]

- Load a pretrained YOLO model and describe its architecture, including the backbone, neck, and detection head.
- Explain the role of each component in the object detection pipeline.

N.B.: Dataloading sometimes may be a concern. Here is a link to the colab notebook, where it has been shown how to do.

<https://colab.research.google.com/drive/1QVeoFZII4k-ot1ioBiDn5L84jrb4tiLN?usp=sharing>

Task B: Dataset Preparation [5 Mark]

- Use a COCO128 dataset (a reduced version of COCO).
- Describe:
 - a) The dataset selection process
 - b) Class distribution and any imbalance
 - c) Preprocessing and annotation format used

Task C: Fine-Tuning Experiments [12 Mark]

- Implement and train the following three configurations:
 - a) Fine-tuning only the final layer (freeze all earlier layers)
 - b) Fine-tuning the last two layers of the network
 - c) Fine-tuning the entire model (no layers frozen)

Task D: Performance Evaluation [10 Mark]

- Evaluate and compare the three models using:
 - a) mAP@0.5
 - b) mAP@0.95
 - c) Training time
 - d) Loss convergence behavior

Task E: Comparative Analysis [8 Mark]

- Compare the performance of the three approaches and discuss:
 - a) Differences in accuracy
 - b) Differences in training time
 - c) Trade-offs between computational cost and performance

Task F: Observations [10 Mark]

- Explain why freezing layers affects model performance. Discuss in terms of:
 - a) Feature extraction
 - b) Transfer learning
 - c) Overfitting and underfitting
- Identify and analyze failure cases for each model, including:
 - a) Incorrect classifications
 - b) Poor localization
 - c) Missed detections
- Provide examples and reasoning.

**** If running the code is becoming delayed focused on solving the remaining questions for the time being**

Section-C [50]

[Clarification Link (for dataset and related queries):

<https://colab.research.google.com/drive/1mAmqHbpltyOhVA4FtBSwpTT0VNpUnSaA?usp=sharing>]

Part A: Theoretical Foundations & Problem Context

In this examination, you are tasked with developing a Multi-Task Learning (MTL) framework for disaster relief using the Dubai Aerial Imagery Dataset. The goal is to simultaneously perform **Semantic Segmentation** (identifying land cover) and **Super-Resolution** (reconstructing structural details).

1. The Data & Task Definitions

- **Input (X):** 128x128 RGB images (Downsampled).
- **Target 1 (Y_{seg}):** 128x128 Class masks ($x = 16,384$ total pixels).
- **Target 2 (Y_{sr}):** 512x512 RGB images ($y = 262,144$ total pixels).

2. Architectural Constraints: The Bottleneck

A shared encoder must compress input data into a universal feature set. A **Bottleneck Architecture** restricts information flow by reducing the channel dimension in the latent space

(e.g., to 32 channels). This forces the model to discard pixel-level noise in favor of high-level semantic abstractions.

3. Conflict Resolution: Gradient Surgery (PCGrad)

During training, task gradients $g_{\{seg\}}$ and $g_{\{sr\}}$ may conflict. If their cosine similarity is negative, they interfere, leading to suboptimal convergence. PCGrad resolves this by projecting a gradient onto the normal plane of the other:

$$g_{seg}^{PCGrad} = g_{seg} - \frac{g_{seg} \cdot g_{sr}}{\|g_{sr}\|^2} g_{sr}$$

4. Task Balancing: Static vs. Adaptive Weighting

MTL frameworks often suffer from "**Seesaw Optimization**"—a scenario where improvements in one task lead to a performance drop in the other, much like a playground seesaw.

- **Static Per-Pixel Scaling:** Normalizes loss by total pixel contribution ($1/x$ and $1/y$). This ensures the "average opinion" of a pixel in either task has equal weight.
- **Adaptive Uncertainty Weighting:** A Bayesian approach using uncertainty. By treating task weights as learnable parameters σ , the model automatically balances the loss:

$$L_{total}(W, \sigma_1, \sigma_2) \approx \frac{1}{2\sigma_1^2} L_{seg}(W) + \frac{1}{2\sigma_2^2} L_{sr}(W) + \log \sigma_1 + \log \sigma_2$$

5. Segmentation: Adaptive Boundary Weighting

Building boundaries are more critical than interior regions in disaster relief. A **Boundary Weight Map** (via Sobel/Laplacian kernels) assigns higher loss weights (e.g., 5.0) to edges to prioritize precision where structures meet.

6. Reconstruction Quality: Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right)$$

Part B: Laboratory Tasks

[General instruction: compute IoU and PSNR after every training run]

Section I: Architecture & Representation (16 Marks)

- **[Code - 8 Marks]** Implement the `SatelliteEncoder` class. Include a backbone of 3-→ 64 -→ 128 -→ 256 channels and a **Bottleneck** layer where the latent feature map channel dimension is reduced to exactly **32 channels**. (The complete architecture is given in the end) Perform baseline training.
- **[Plot - 4 Marks]** Visualize activations: Compare an early "Edge-Detection" layer vs. the "Bottleneck" layer for a single image.
- **[Analysis - 4 Marks]** Explain the logic: How does a bottleneck force the encoder to learn "Semantic" features (like identifying a building) rather than just "memorizing" pixel intensities?

Section II: Optimization Electives (24 Marks)

Instructions: Choose **ANY TWO** of the following three tracks to implement and analyze. Each track is worth **12 marks**. (unless stated otherwise, the question implies end to end training)

- **Track A: Gradient Surgery (PCGrad)**
 - **[Code - 6 Marks]** Implement `resolve_conflict(g_seg, g_sr)` and integrate it into your training loop.
 - **[Analysis - 6 Marks]** Plot Gradient Similarity vs. Training Steps. Explain how this projection prevents the tasks from "fighting" over the shared weights.
- **Track B: Weighting Strategies (Static vs. Adaptive)**
 - **[Code - 6 Marks]** Implement both **Static Scaling** ($1/x$, $1/y$) and **Uncertainty Weighting** (learnable parameters).
 - **[Analysis - 6 Marks]** Plot the training loss curves. Explain why Uncertainty Weighting is more robust when one task (SR) has 16x the pixel density of the other.
- **Track C: Adaptive Boundary Weighting**
 - **[Code - 6 Marks]** Generate a `BoundaryWeightMap` function (Weights: Edges=5.0, Interior=1.0) and incorporate it into the Segmentation Loss.
 - **[Analysis - 6 Marks]** Visualize the predicted masks. Explain why forcing the model to focus on boundaries is more effective for disaster mapping than standard Cross-Entropy.

Section III: Qualitative Evaluation & Discussion (10 Marks)

- **[Analysis - 5 Marks]** Based on your results, did your chosen elective tracks achieve **Positive Transfer** (where learning Task A actually helps Task B)? Provide evidence from your mIoU and PSNR metrics.
- **[Discussion - 5 Marks]** Describe the **Seesaw Optimization** observed during your runs. Which task was the "bully" initially, and how did your chosen strategies help stabilize the shared representation?

Part C: Implementation Specifications

1. Architecture Specs

- **Encoder:** 3->64->128->256 (Max-pooling after each block).
- **Bottleneck:** 256 -> 32 at 16x16 spatial resolution.
- **Heads:**
 - **Segmentation:** Decoder upsampling to 128 x 128.
 - **SR:** PixelShuffle layers upsampling to 512 x 512 (or the native resolution of the hr data).

2. Hyperparameters

Parameter	Value
Learning Rate (\$\eta\$)	1×10^{-4}
Optimizer	Adam
Batch Size	8
Epochs	20 (minimum)

Submission Checklist:

- [] **SatelliteEncoder** with 32-channel bottleneck.
- [] Implementation of **TWO** chosen Elective Tracks from Section II.
- [] Qualitative analysis of Positive Transfer and Seesaw Dynamics.
- [] Visualization plots for Activations and Task Outputs.